RAG & Model Context Protocol (MCP)

7 pages • document ingestion, chunking, embeddings, retrieval, generation and MCP integration

1. Retrieval-Augmented Generation (RAG)

Definition

Retrieval-Augmented Generation (RAG) combines information retrieval with LLM generation. The application retrieves relevant external content and supplies it to the model as context.

Why RAG?

Use private or domain-specific documents, incorporate updated information, provide source references and avoid retraining whenever the knowledge base changes.

Basic pipeline

Ingestion → cleaning and chunking → embeddings and indexing → query embedding → retrieval → context assembly → LLM generation → evaluation.

Ingestion

Load source documents such as PDFs, websites, text files or databases.

Indexing

Split content into chunks, create embeddings and store vectors with text and metadata.

Retrieval

Find chunks relevant to the user's query, often using vector similarity, keyword search or both.

Generation

Send the query and retrieved evidence to the LLM with instructions to answer from the context.

RAG vs fine-tuning

RAG supplies external context at query time. Fine-tuning updates model parameters and adapts behaviour or task patterns.

Key takeaway: RAG does not permanently teach facts to the model; it provides relevant information at inference time.

2. Document Loading & Chunking

Document loaders

Loaders convert source content into document objects, commonly containing page_content and metadata such as source filename or page number.

PDF loaders

PyPDFLoader commonly loads PDF content page by page. PyMuPDFLoader uses PyMuPDF for PDF text and metadata extraction.

Inspect extraction

Check text quality before indexing. Scanned PDFs may require OCR; tables and multi-column layouts may need special handling.

Why chunk?

LLMs have context limits. Chunking makes retrieval more precise and helps supply only relevant passages.

RecursiveCharacterTextSplitter

Tries separators in sequence—often paragraphs, lines, spaces and finally characters—to preserve meaningful text boundaries.

chunk_size

Target maximum chunk length, measured in characters by default for RecursiveCharacterTextSplitter.

chunk_overlap

Text repeated between adjacent chunks to reduce loss of context at boundaries.

Example

from langchain_text_splitters import RecursiveCharacterTextSplitter\nsplitter = RecursiveCharacterTextSplitter(chunk_size=800, chunk_overlap=120)\nchunks = splitter.split_documents(documents)

Key takeaway: Too-large chunks include irrelevant material; too-small chunks may lose context. Tune using real retrieval tests.

3. Embeddings & Vector Stores

Embeddings

An embedding model converts text into a dense numerical vector. Semantically similar text often maps to nearby vectors.

Similarity measures

Cosine similarity compares vector angle; dot product measures alignment and magnitude; Euclidean distance measures straight-line distance.

Vector store/database

Stores embeddings with chunk text and metadata, and supports similarity search. Examples: FAISS, Chroma, Pinecone, Weaviate and Qdrant.

Indexing workflow

Split documents → embed each chunk → store vector, text and metadata → embed query → search similar vectors.

Metadata

May include source, page, date, category, tenant or permissions. Filters narrow retrieval and help enforce access boundaries.

Top-k

Controls how many candidate chunks are returned. More chunks are not always better; tune k and thresholds.

Embedding compatibility

Document and query embeddings should be compatible—typically the same model or a pair designed for the same vector space.

Key takeaway: Preserve source and page metadata so retrieved passages can be traced and cited.

4. Retrieval Strategies & Context Assembly

Dense retrieval

Uses embeddings to find semantically related passages, even when the wording differs.

Sparse retrieval

Keyword methods such as BM25 are strong for exact terms, identifiers and names.

Hybrid retrieval

Combines dense semantic search with sparse keyword search.

Reranking

A reranker scores query-document pairs more precisely and reorders initial candidates. Retrieve a broader set, then keep the strongest passages.

Query rewriting

Rewrites ambiguous user questions into search-friendly queries.

Multi-query retrieval

Creates multiple search formulations to improve coverage.

HyDE

Creates a hypothetical answer/document and uses its embedding to find related real documents.

Query decomposition

Splits a complex question into smaller sub-questions.

Context assembly

Remove duplicates, preserve source IDs, order evidence logically, fit the context budget and clearly delimit retrieved text as untrusted reference content.

Key takeaway: Retrieval precision and context quality matter as much as the number of retrieved chunks.

5. Generation, Evaluation & Troubleshooting

Grounded generation

Instruct the model to answer using supplied context, cite sources and say when the context does not contain the answer.

Retrieval evaluation

Did the system retrieve the right evidence? Consider precision/recall at k, ranking quality and coverage.

Generation evaluation

Is the answer correct, relevant and supported by retrieved evidence? Assess unsupported claims and citation accuracy.

System evaluation

Track latency, cost, reliability, failure rates and user outcomes.

PDF extraction failure

Tables, columns or scanned pages may be extracted poorly; inspect source pages and use OCR or layout-aware extraction when needed.

Chunking failure

Relevant facts may be split or mixed with unrelated text; tune chunk size, overlap and boundaries.

Retrieval failure

Try query rewriting, metadata filters, hybrid retrieval, reranking or a different embedding model.

Hallucination

Strengthen grounding instructions, require evidence and evaluate unsupported statements.

Checklist

Inspect extracted text; retain source metadata; test representative queries; evaluate retrieval separately from generation; protect permissions and sensitive data.

Key takeaway: A RAG system is only as reliable as its source data, extraction, retrieval and grounding steps.

6. Model Context Protocol (MCP) Fundamentals

Definition

Model Context Protocol (MCP) is an open protocol that standardizes how AI applications connect to external tools and data sources.

Host

The AI application the user interacts with; it coordinates model interaction and MCP connections.

MCP client

Manages a connection to an MCP server and exchanges protocol messages.

MCP server

Exposes capabilities such as tools, resources and prompts.

External system

A database, API, filesystem or service used by the MCP server.

Tools

Executable functions the model can request, such as search, calculations or sending a message.

Resources

Data or content the client can read, such as files or records.

Prompts

Reusable prompt templates exposed by a server.

Typical interaction

Client connects and initializes → discovers capabilities → invokes a tool or reads a resource → receives the result.

Key takeaway: MCP standardizes how capabilities are exposed; it does not itself decide which tool an LLM should use.

7. MCP Transports, LangChain & Security

stdio transport

The client launches a local server process and exchanges protocol messages over standard input/output. Common for local tools.

Streamable HTTP

The client communicates with a server over HTTP. Useful for separately hosted or remote services.

LangChain adapter

MultiServerMCPClient can connect to multiple MCP servers and expose discovered tools to a LangChain agent.

Client example

from langchain_mcp_adapters.client import MultiServerMCPClient\nclient = MultiServerMCPClient({\n 'math': {'command': sys.executable, 'args': [r'D:/project/mathserver.py'], 'transport': 'stdio'},\n 'weather': {'url': 'http://127.0.0.1:8000/mcp', 'transport': 'streamable_http'}\n})\ntools = await client.get_tools()

stdio connection errors

Connection closed may mean the server crashed, the path is wrong, dependencies are missing, the wrong interpreter is used or stdout contains debug prints.

HTTP connection errors

Check that the server is running, URL and port are correct, and the configured transport matches the server.

Windows paths

Use raw strings such as r'D:\\folder\\server.py' or forward slashes to avoid escape-sequence warnings.

Security

Expose only necessary tools; validate arguments; use authentication and least privilege; require approval for sensitive actions; treat tool output as untrusted; never log secrets.

RAG vs MCP

RAG retrieves knowledge and supplies context. MCP connects an AI application to external capabilities, including tools and data. They can be used together.

Key takeaway: RAG helps an agent access relevant knowledge; MCP helps an agent connect to external systems and use their capabilities.